

MBSD Assignment #3 A.Y. 2024/25

Purposes

- Perform some parts of the Functional and Technical Safety Concept analysis, according to ISO26262, of a “one pedal controller” for a car.
- Implement some of the safety concepts in the Simulink model of the controller developed in Laboratory #2.
- Perform unit and integration tests on the implemented safety-related functionalities.

It is available an example of a Functional Safety Concept for the item Front Light Manager (FLM).

The deliverable, composed of

- the report (the following pages of this document)
- the Simulink models on where the safety concepts have been implemented
- all the needed files to replicate the software testing results

has to be provided as a .ZIP file (do not use other compression formats) up to **May 14th at 23:59**. It shall also contain a brief report explaining the design of the controller using the following template.

It is sufficient that only one of the group members uploads it.

IMPORTANT:

For making the Functional Safety Concept coherent between the groups of the course, consider as ASIL C all the safety goals related to unintended acceleration (those leading to an increase of the vehicle's speed modulus) and as ASIL B the warnings to the driver and the unintended deceleration (those leading to a decrease of the vehicle's speed modulus).

Model-Based Software Design, A.Y. 2022/23

Laboratory 3 Report

Components of the working group (max 2 people)

- Name Surname, student ID

Federica D'Ugo, 345772

Francesca Fornasier, 338688

Functional Safety Concept

One pedal

Functional safety architecture

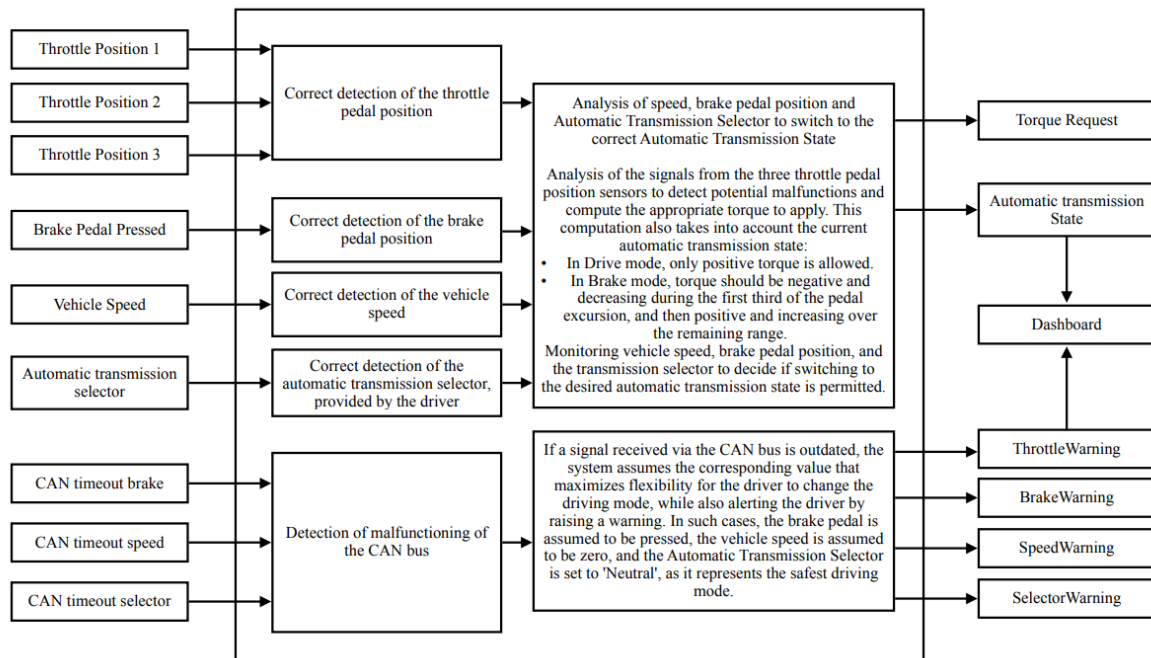


Figure 1 Functional safety architecture

Attributes of the Safety Goals

Fill the table with the safety goal descriptions

SG 1	The item should avoid unintended breaking
SG 2	The item should avoid unintended acceleration
SG 3	The item should avoid insufficient acceleration
SG 4	The item should avoid insufficient breaking
SG 5	The item should avoid motion in incorrect direction

Fill in the attribute/parameters of the safety goal

Safety goal	Attributes/Parameters of the safety goal				
	Integrity (ASIL)	Safe state	Fault tolerance time	Warning concept	Degradation concept
SG 1	B	The item should switch to D mode	0.5/1 seconds	The driver must be warned through the dashboard	Loss of regenerative breaking
SG 2	C	The system should set the torque to zero and	0.5/1 seconds	The driver must be warned	Lost all functionalities

		switch to Neutral mode		through the dashboard	
SG 3	B	The system should inform the driver of the insufficient acceleration issue through the dashboard	1/2 seconds	The driver must be warned through the dashboard	
SG 4	B	The system sets the torque to zero and switches to Neutral mode	1/2 seconds	The driver must be warned through the dashboard	
SG 5	B	The system requests zero torque	0.5 seconds	The driver must be warned through the dashboard	

Functional (and technical) safety requirements and allocation

		Define functional safety requirements		Allocation of requirements on systems and elements	
		Safety requirements	Remark	If applicable, allocate the safety requirements to other Items / Systems	If applicable, allocate the safety requirements to equipment other technologies to minimize risk. That could be e.g. hydraulic, mechanical equipment
Safety goals	Avoid Unintended acceleration	Check if the measure from the throttle pedal is valid	no	no	no
		Check if the communication with the automatic transmission selector state	no	no	no

		Check if the communication with vehicle speed sensor is working	no	no	no
		Check if the torque request computation is valid	no	no	no
		Check if the communication with the dashboard is working	no	no	no
	Avoid Unintended breaking	Check if the communication with the breaking pedal is working	Not safety relevant	no	You can still use mechanical emergency brakes

We chose not to explicitly list the remaining safety goals, as their corresponding safety requirements are already covered by those previously defined in the report.

ASIL preliminary architecture¹

The purpose of this figure is to assign ASILs to the components indicated in Figure 1 on the functional safety architecture.

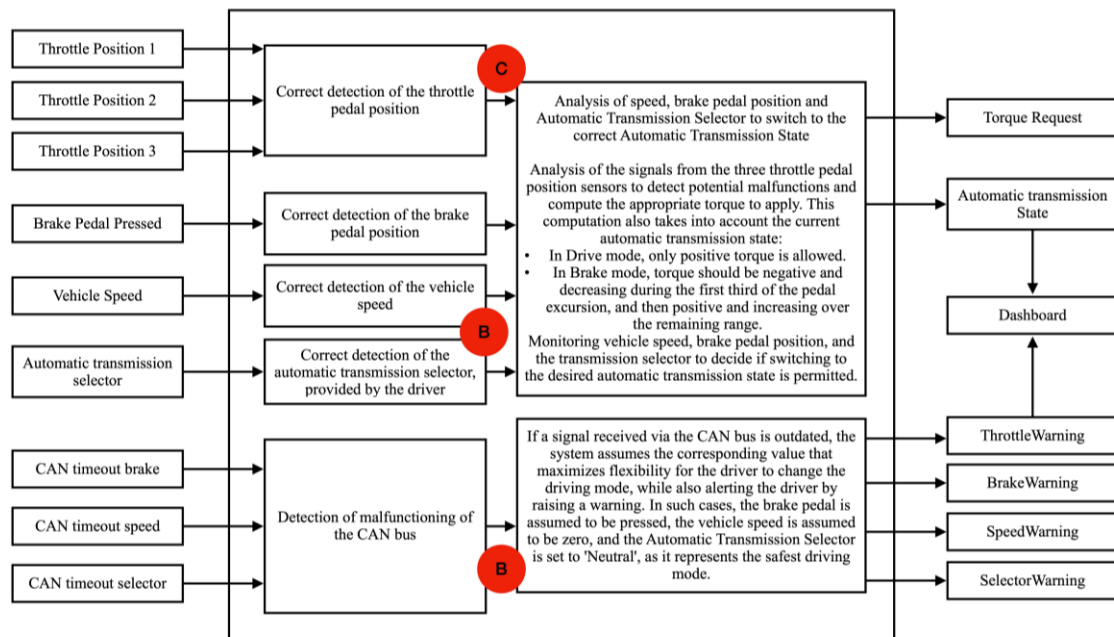


Figure 2 Preliminary architecture (without ASIL decomposition).

¹ See document 02-iso26262.pdf, slides 89, 90, 91, 92, 93.

Brief report explaining the design of the controller

In this laboratory activity, we enhanced the controller by adding several MATLAB functions aimed at improving signal reliability and system safety:

- Three MATLAB Function blocks were implemented to verify the validity of the inputs used by the finite state machine (BrakePedalPressed, VehicleSpeed_kmh, and AutomaticTransmissionSelectorState). Each function checks the corresponding CAN timeout boolean to ensure the data is current and not outdated.
- One MATLAB Function was developed to process the three redundant throttle pedal signals. It calculates the average of the values that fall within a specified tolerance of ± 0.05 , ensuring consistency among the sensors.
- Another MATLAB Function sets the torque command to zero if a reliable average cannot be determined (e.g., due to excessive disagreement between the signals).
- In the event of a fault in any of these inputs, the system generates a warning specific to the affected signal, allowing for more accurate fault identification and easier diagnostics

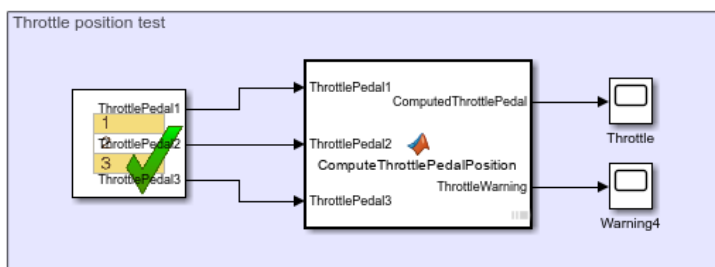
All of these implementations will be described in detail in the following pages.

Implementations²

Functional redundancies

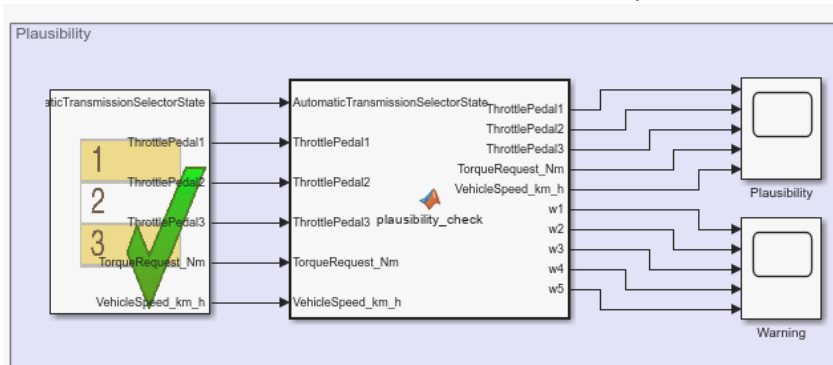
Indicate which components of the functional safety architecture are implemented with redundancies.

- To verify the correctness of the throttle pedal position, we decided to use three sensors that measure its position independently. We implemented a MATLAB function to check whether each of the three measurements is valid (imposing a tolerance of 0.05). If all three values are correct, the function calculates their average. If only some of the values are valid, the output is computed as the average of the valid measurements. If one or more channels are faulty, a warning message is sent to indicate which channels are incorrect. If two channels are faulty, it is not possible to compute the average; therefore, the throttle pedal position is set to -1.



Implemented plausibility checks

Indicate which signals are subjected to plausibility checks. Indicate also the checks needed for the channels with redundancies in place.



- To ensure the reliability of the throttle pedal position measurements, we implemented a validation check on the data provided by the three sensors (ThrottlePedal1, ThrottlePedal1, ThrottlePedal3). Each measurement is verified to confirm it falls within the valid range of 0 to 1. Any value outside this range is considered implausible and is replaced with a value of -1 for the first pedal, -3 for the second and -5 for the third, which lies well outside the acceptable

² In the ISO26262 the implementations are based on a document called *Technical Safety Concept*, but for simplicity we move straight from the *Functional Safety Concept* to software implementations. A guideline for the implementation phase can be found in the document 02-iso26262.pdf from slide 81, in particular slide 86.

tolerance and clearly distinguishes it from valid readings and other system components.

```
function [ThrottlePedal1,ThrottlePedal2,ThrottlePedal3,w1,w2,w3] = plausibility_check(ThrottlePedal1,ThrottlePedal2,ThrottlePedal3)
    w1=single(0);
    w2=single(0);
    w3=single(0);
    if(ThrottlePedal1<0 || ThrottlePedal1>1)
        ThrottlePedal1=single(-1);
        w1=single(1);
    end
    if(ThrottlePedal2<0 || ThrottlePedal2>1)
        ThrottlePedal2=single(-3);
        w2=single(1);
    end
    if(ThrottlePedal3<0 || ThrottlePedal3>1)
        ThrottlePedal3=single(-5);
        w3=single(1);
    end
end
```

- Additional plausibility checks were implemented on the torque request signal. Specifically, when the vehicle is in reverse driving mode, the torque request must be within the range of -40 to 0 Nm, as positive values would be incompatible with reverse operation.

In all other driving modes, the valid range is -80 to 80 Nm.

If the torque request exceeds the allowed range, it is forced to zero, as specified in the safety requirements for ASIL C.

```
function [w4,TorqueRequest_Nm] = checkplausibilitytorque(TorqueRequest_Nm,AutomaticTransmissionSelectorState)
    w4=single(0);
    if (AutomaticTransmissionSelectorState==TransmissionState.Reverse)
        if (TorqueRequest_Nm<-40 || TorqueRequest_Nm>0)
            TorqueRequest_Nm=single(0);
            w4=single(1);
        end
    else if(TorqueRequest_Nm<-80 || TorqueRequest_Nm>80)
        TorqueRequest_Nm=single(0);
        w4=single(2);
    end
end
```

- A similar plausibility check was applied to the vehicle speed, which is required to remain within the range of -60 km/h to 240 km/h, encompassing both reverse operation and the maximum permissible forward speed. If the speed value falls outside this range, it is set to zero.

```
function [w5,VehicleSpeed_kmh] = speedplausibilitycheck(VehicleSpeed_kmh)
    w5=single(0);
    if(VehicleSpeed_kmh<-60 || VehicleSpeed_kmh>240)
        VehicleSpeed_kmh=single(0);
        w5=single(1);
    end
end
```

- It was not possible to perform validity checks on the brake pedal value, as it is a Boolean signal with a range limited to either true or false (0 or 1). Similarly, no range check was applied to the automatic transmission selector state, since it is represented as a string corresponding to predefined driving modes.

In all analyzed cases where a measurement fell outside its valid range, a warning was generated in addition to the applied corrective action (with signals w1, w2, w3, w4, w5 displayed on the scope).

To ensure completeness, we performed a coverage test to assess whether all relevant scenarios were taken into account:

1. MATLAB Function "[MATLAB Function](#)"

Metric	Coverage (this object)	Coverage (inc. descendants)
Cyclomatic Complexity	1	20
Decision	NA	100% (15/15) decision outcomes
Condition	NA	100% (24/24) condition outcomes
MCDC	NA	100% (12/12) conditions reversed the outcome
Execution	NA	NA

MATLAB Function "[plausibility_check](#)"

[Justify or Exclude](#)

Parent: [test1/MATLAB Function](#)

Metric	Coverage
Cyclomatic Complexity	19
Condition	100% (24/24) condition outcomes
Decision	100% (15/15) decision outcomes
MCDC	100% (12/12) conditions reversed the outcome

Code Coverage Evaluation

The results show that the `plausibility_check` achieves full structural coverage:

- Decision Coverage: 100% (15/15 decision outcomes)
- Condition Coverage: 100% (24/24 condition outcomes)
- MCDC (Modified Condition/Decision Coverage): 100% (12/12 conditions that independently affect decision outcomes)

This indicates that all logical paths have been evaluated during testing, and each condition has been shown to independently influence the outcome, fulfilling the criteria for safety-critical software.

Software testing

Implemented unit tests

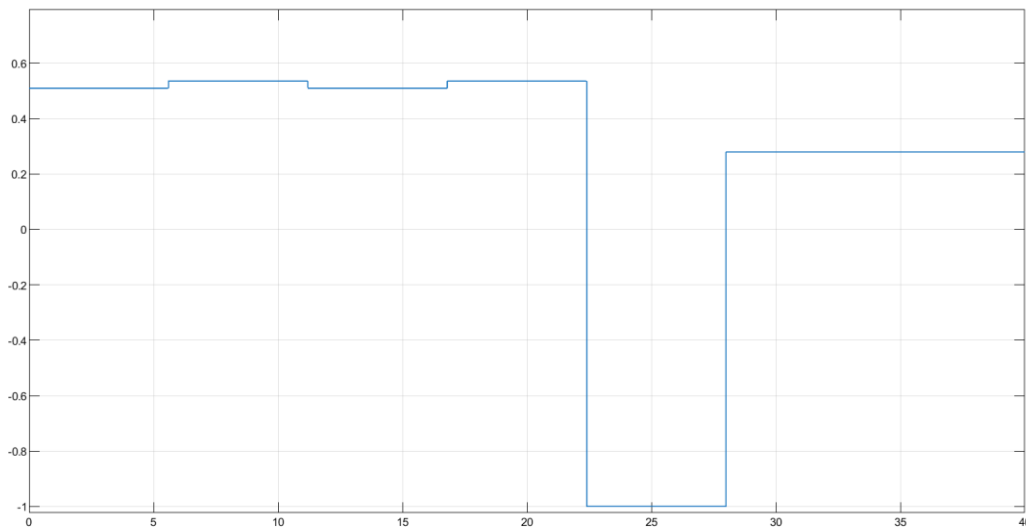
Describe in English the test performed to verify the correct functionality of the safety mechanism implemented.

- The first unit test focused on the throttle pedal position. We had three channels providing this information: `data1`, `data2`, and `data3`. A tolerance of 0.05 was set to verify the consistency of the data. To determine if a channel is reliable, we compare its value with the other two. If the absolute difference between the values is less than the tolerance, the data from that channel is considered valid. Otherwise, it is marked as false and excluded from the computation of the average throttle position.

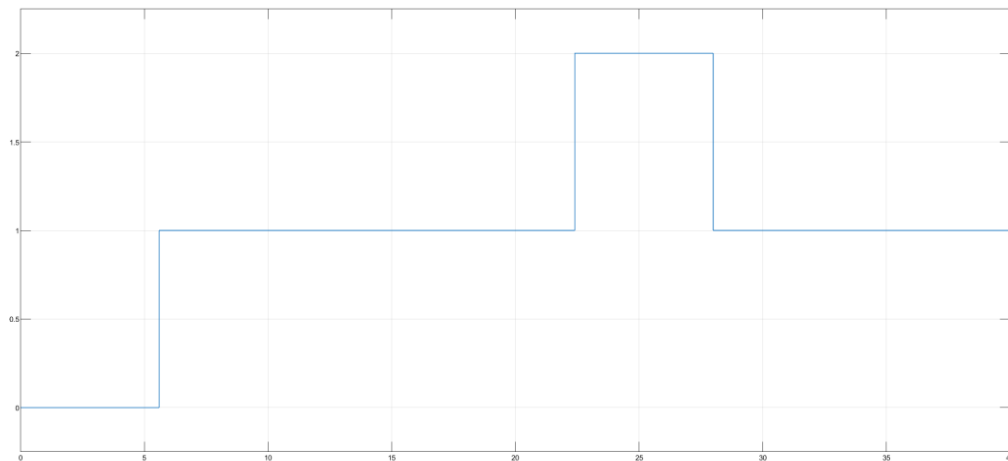
```
function [ComputedThrottlePedal,ThrottleWarning] = ComputeThrottlePedalPosition(ThrottlePedal1, ThrottlePedal2, ThrottlePedal3)
    tol = 0.05;
    ThrottleWarning=single(0);
    if(abs(ThrottlePedal1-ThrottlePedal2)<tol)
        if(abs(ThrottlePedal1-ThrottlePedal3)<tol && abs(ThrottlePedal2-ThrottlePedal3)<tol)
            ComputedThrottlePedal = mean([ThrottlePedal1 ThrottlePedal2 ThrottlePedal3]);
        else
            ComputedThrottlePedal = mean([ThrottlePedal1 ThrottlePedal2]);
            ThrottleWarning = single(1);
        end
    elseif(abs(ThrottlePedal1-ThrottlePedal3)<tol)
        ComputedThrottlePedal = mean([ThrottlePedal1 ThrottlePedal3]);
        ThrottleWarning = single(1);
    elseif(abs(ThrottlePedal2-ThrottlePedal3)<tol)
        ComputedThrottlePedal = mean([ThrottlePedal2 ThrottlePedal3]);
        ThrottleWarning = single(1);
    else
        ThrottleWarning = single(2);
        ComputedThrottlePedal = single(-1);
    end
end
```

The table below presents a summary of the test results, obtained by executing the function with different sets of input values.

Test	Data1	Data2	Data3	Expected output	Real output	Expected Warning	Real warning
Test 1	0,5	0,52	0,51	0,51	0,51	All the channels are correct)	0
Test 2	0,55	0,52	0,5	0,535	0,535	Warning: One channel is incorrect (Data3).	1
Test 3	0,5	0,55	0,52	0,51	0,51	Warning: One channel is incorrect (Data2).	1
Test 4	0,7	0,55	0,52	0,535	0,535	Warning: One channel is incorrect (Data1)	1
Test 5	0,5	0,55	0,45	(not possible)	-1	Warning: All the channels are incorrect	2
Test 6	0,3	0,26	0,34	0,32	0,32	Warning: One channel is incorrect (data2)	1



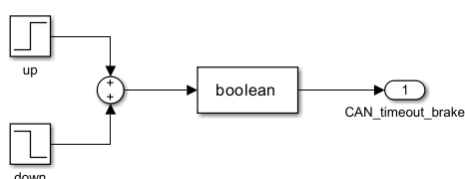
Throttle pedal position values during the test simulations. Each test lasts approximately 5 seconds (x axis in s). The value -1 means that we can't compute the average since 2 channels are incorrect.



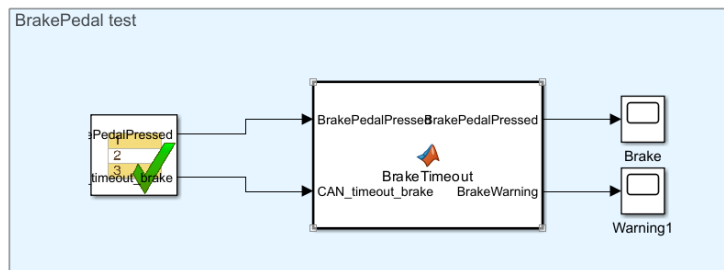
Warning signal during the test simulations. Each test lasts approximately 5 seconds (x-axis in seconds). A warning value of 0 indicates that all channels are correct; 1 means that one channel is incorrect; 2 indicates that two channels are incorrect, making it impossible to compute the average.

To determine which channel is correct or incorrect, it is important to observe the order in which the comparisons are made. In our case, channels 1 and 2 are compared first, followed by 1 and 3, and finally 2 and 3, as demonstrated by the results of tests 2 and 3.

To ensure that the information regarding the brake pedal, the automatic transmission selector state, and the vehicle speed is up to date when received by the controller, we implemented CAN timeouts. These are represented by boolean values that change based on a step change (0=false and 1=true). When a CAN timeout value is true, it indicates that the data being received by the controller is outdated.

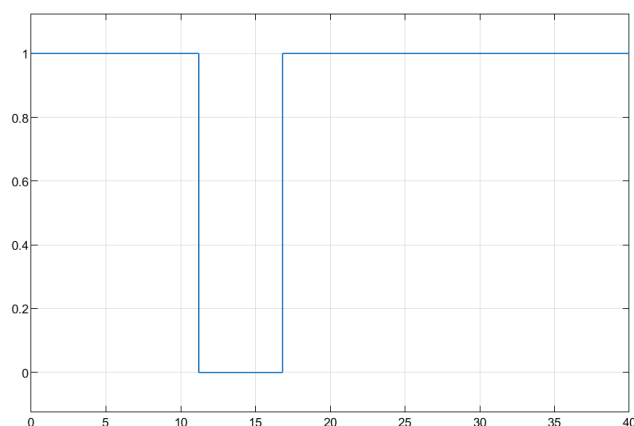


- The second unit test focused on the brake pedal. The validity of the data is verified using the CAN timeout signal. When the CAN signal is true, it means the data is outdated, so we always assume the brake pedal is pressed (value 1=True). In this case, the driver is still allowed to change the driving mode, but a warning is issued. When the signal is false, the data is up to date and can be used reliably (the brake pedal value can be either 0=False or 1=True, depending on the actual input):

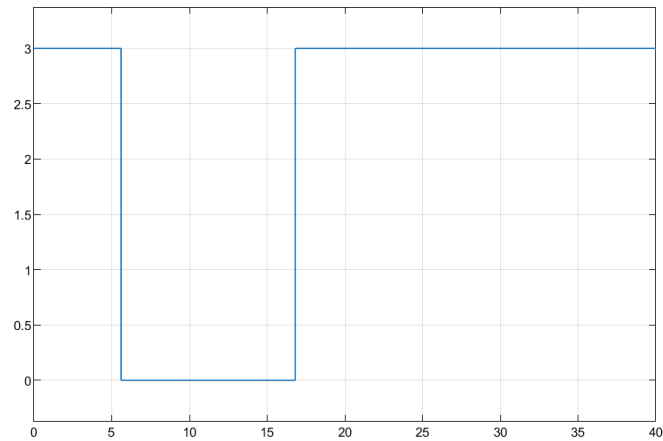


```
function [BrakePedalPressed, BrakeWarning] = BrakeTimeout(BrakePedalPressed, CAN_timeout_brake)
    if(CAN_timeout_brake==true)
        BrakePedalPressed=true;
        BrakeWarning=single(3);
    else
        BrakeWarning=single(0);
    end
end
```

Test	CAN_timeout_brake	BrakePedalPressed	Expected output of brakePedal Pressed	Real output	Expected Warning	Real warning
Test 1	True	True (=pressed)	True	True	The data is outdated	3
Test 2	False	True	True	True	The data is current	0
Test 3	False	False (=not pressed)	False	False	The data is current	0
Test 4	True	False	True	True	The data is outdated	3

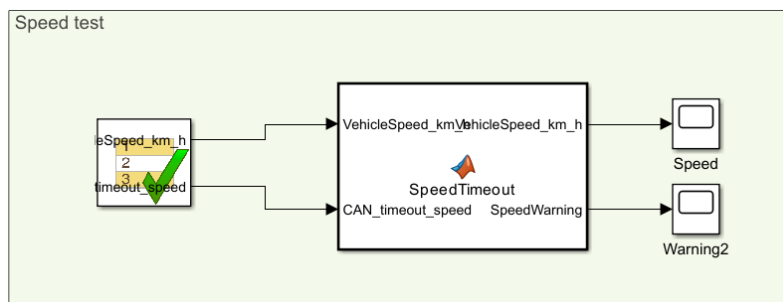


Brake pedal position values during the test simulations. Each test lasts approximately 5 seconds (x axis in s).



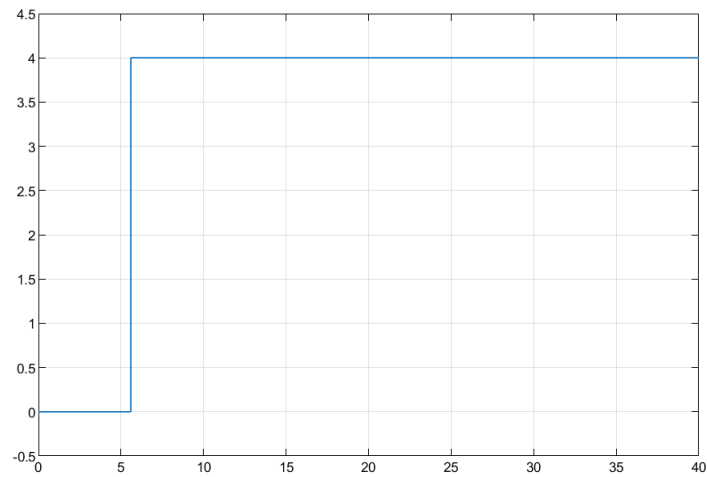
Warning signal during the test simulations. Each test lasts approximately 5 seconds (x-axis in seconds). A warning value of 0 indicates that the data is current, instead a value of 3 means that the data is outdated.

- The third unit test focused on vehicle speed. Data validity is verified via the CAN timeout signal: if true, the data is outdated and the speed value is assumed to be 0; if false, the data is current and reflects the actual input from the vehicle speed sensor.

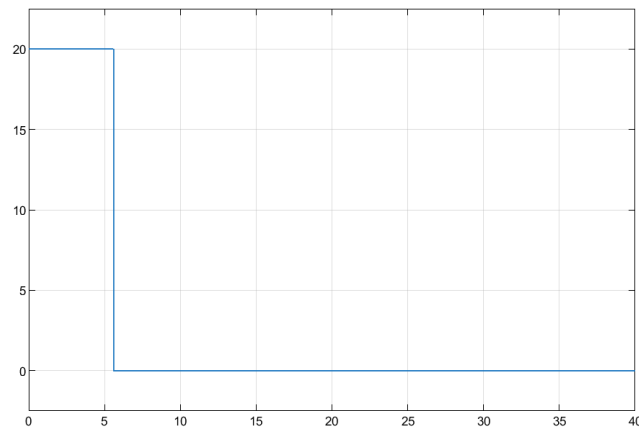


```
function [VehicleSpeed_kmh, SpeedWarning]= SpeedTimeout(VehicleSpeed_kmh, CAN_timeout_speed)
    if(CAN_timeout_speed==true)
        VehicleSpeed_kmh=single(0);
        SpeedWarning=single(4);
    else
        SpeedWarning=single(0);
    end
end
```

	CAN_timeout_speed	VehicleSpeed_kmh	Expected output of VehicleSpeed_kmh	Real output	Expected Warning	Real warning
Test 1	False	20	20	20	The data is current	0
Test 2	True	50	0	0	The data is outdated	4

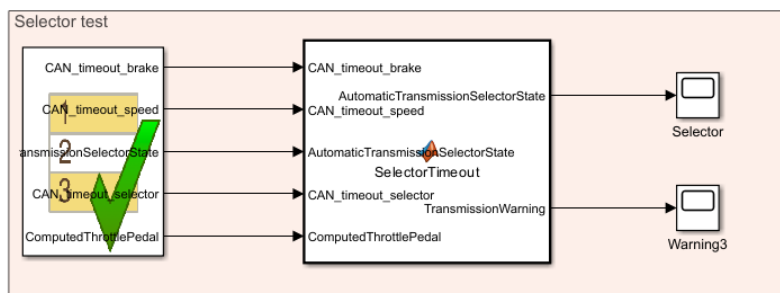


VehicleSpeed_kmh values during the test simulations. Each test lasts approximately 5 seconds (x axis in s).



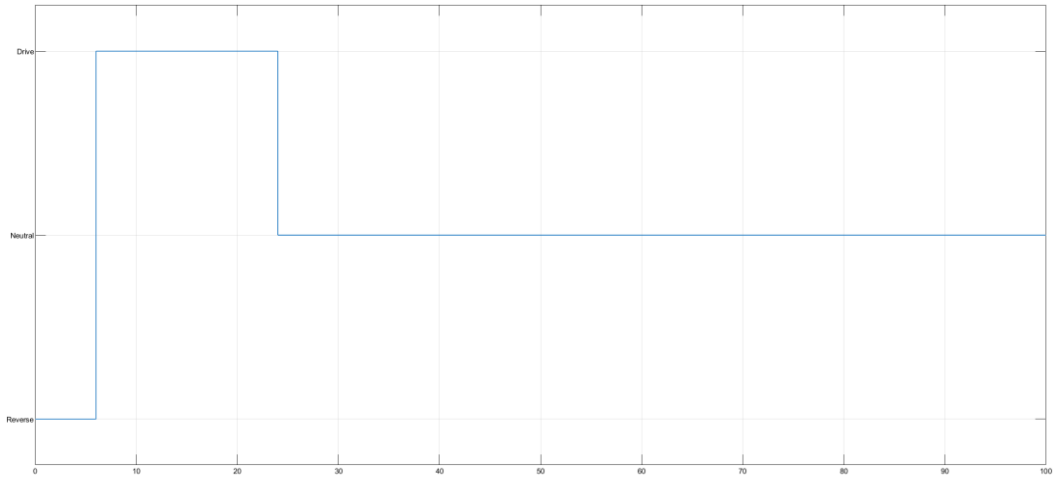
Warning signal during the test simulations. Each test lasts approximately 5 seconds (x-axis in seconds). A warning value of 0 indicates that the data is current, instead a value of 4 means that the data is outdated

- The fourth unit test focused on the selector logic. Data validity is assessed through the CAN timeout signal. If the signal is active (i.e., true), it indicates that the data is outdated; in this case, the system automatically switches to drive mode, as specified by the ASIL B requirements. Conversely, if the signal is inactive (i.e., false), the data is considered valid and accurately reflects the selector input.

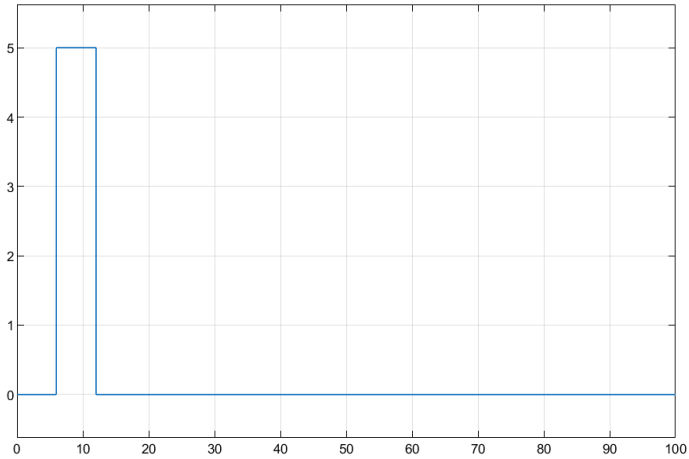


```
function [AutomaticTransmissionSelectorState, TransmissionWarning] = SelectorTimeout(CAN_timeout_brake, CAN_timeout_speed, AutomaticTransmissionSelectorState, CAN_timeout_selector,
if(CAN_timeout_selector==true)
    TransmissionWarning = single(5);
    AutomaticTransmissionSelectorState = TransmissionState.Drive;
else
    TransmissionWarning = single(0);
end
if(CAN_timeout_speed==true || CAN_timeout_brake==true)
    AutomaticTransmissionSelectorState = TransmissionState.Drive;
end
if(ComputedThrottlePedal==single(-1))
    AutomaticTransmissionSelectorState = TransmissionState.Neutral;
end
```

	CAN_timeout_selector	CAN_timeout_speed	CAN_timeout_brake	ComputedThrottlePedal	AutomaticTransmissionSelectorState	Expected output of AutomaticTransmissionSelectorState	Real output	Expected Warning	Real warning
Test 1	False	False	False	1	Reverse	Reverse	Reverse	The data is current	0
Test 2	True	False	False	1	Break	Drive	Drive	The data is outdated	5
Test 3	False	True	False	1	Break	Drive	Drive	The data is current	0
Test 4	False	False	True	1	Break	Drive	Drive	The data is current	0
Test 5	False	False	False	-1	Drive	Neutral	Neutral	The data is current	0



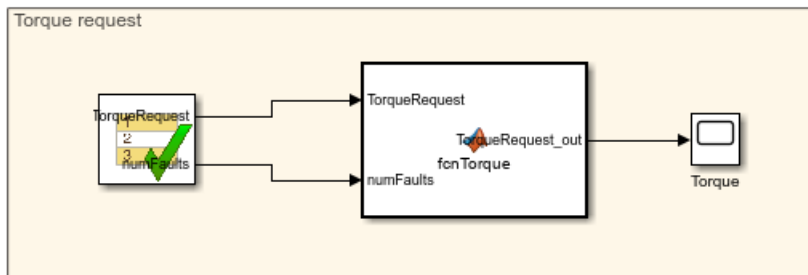
AutomaticTransmissionSelectorState values during the test simulations. Each test lasts approximately 5 seconds (x axis in s).



Warning signal during the test simulations. Each test lasts approximately 5 seconds (x-axis in seconds). A warning value of 0 indicates that the data is current, instead a value of 5 means that the data is outdated

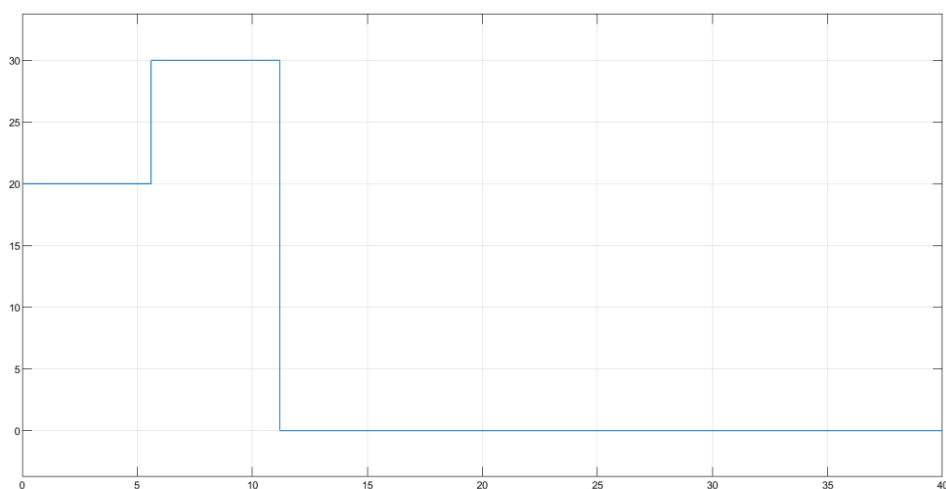
- The final unit test dealt with the torque request, which needs to be forced to zero if errors are found in two or more of the channels used to read the throttle pedal position. This is because the torque calculation relies heavily on accurate input from these channels. The presence of errors is indicated by the numFaults variable.

The key scenario is when numFaults equals 2. This means that two channels are faulty, making it impossible to reliably determine the throttle pedal position by averaging the available values. As a result, the system must set the torque request to zero. In all other cases, when fewer than two faults are detected, the system can still process the input correctly, and no special handling is required.



```
function TorqueRequest_out = fcnTorque(TorqueRequest, numFaults)
    if(numFaults==2)
        TorqueRequest_out = single(0);
    else
        TorqueRequest_out = TorqueRequest;
    end
end
```

	NumFaults	TorqueRequest	Expected output of TorqueRequest	Real output
Test 1	0	20	20	20
Test 2	1	30	30	30
Test 3	2	25	0	0



TorqueRequest_out values during the test simulations. Each test lasts approximately 5 seconds (x axis in s).

For each of these tests, we performed a coverage analysis:

Summary

Model Hierarchy/Complexity		Run 1				MCDC	Execution
		Decision	Condition				
1. test1	84	100% 	100% 	100% 	100% 	100% 	100% 
2. ... MATLAB Function	16	100% 	100% 	100% 	100% 	NA	NA
3. ... MATLAB Function1	3	100% 	NA		NA	NA	NA
4. ... MATLAB Function2	7	100% 	100% 	100% 	100% 	NA	NA
5. ... MATLAB Function3	3	100% 	NA		NA	NA	NA
6. ... MATLAB Function4	3	100% 	NA		NA	NA	NA
7. ... MATLAB Function5	3	100% 	NA		NA	NA	NA
8. ... Test Sequence	11	100% 	NA		NA	100% 	
9. ... SF: Test Sequence	10	100% 	NA		NA	NA	NA
10. ... Test Sequence1	3	100% 	NA		NA	100% 	
11. ... SF: Test Sequence1	2	100% 	NA		NA	NA	NA
12. ... Test Sequence2	3	100% 	NA		NA	100% 	
13. ... SF: Test Sequence2	2	100% 	NA		NA	NA	NA
14. ... Test Sequence3	7	100% 	NA		NA	100% 	
15. ... SF: Test Sequence3	6	100% 	NA		NA	NA	NA
16. ... Test Sequence4	5	100% 	NA		NA	100% 	
17. ... SF: Test Sequence4	4	100% 	NA		NA	NA	NA
18. ... Test Sequence5	19	100% 	NA		NA	100% 	
19. ... SF: Test Sequence5	18	100% 	NA		NA	NA	NA

Coverage analysis

Figure 12 confirms that the test objectives have been fully achieved. As shown in the figure, all relevant components of the model(including associated MATLAB Function blocks and Test Sequences) have reached 100% coverage in terms of decision logic, and where applicable, also for condition and MCDC.

In particular:

- The main model and all subcomponents have been exercised in a way that guarantees complete exploration of decision paths, confirming the correctness and completeness of the logical branches.
- The execution coverage is also 100% for all the test sequences executed, indicating that all parts of the model involved in the test were actually run during the simulation.
- Although some coverage types are marked as "NA" (not applicable), this is due to the simplicity of certain functions or structures that do not include compound conditions or decisions, and therefore do not require condition or MCDC analysis.

These results demonstrate that the unit test is comprehensive and effectively validates the behavior of the model under all expected conditions. No uncovered logic or untested paths remain within the scope of this test, supporting a high level of confidence in the correctness of the implemented functionality.

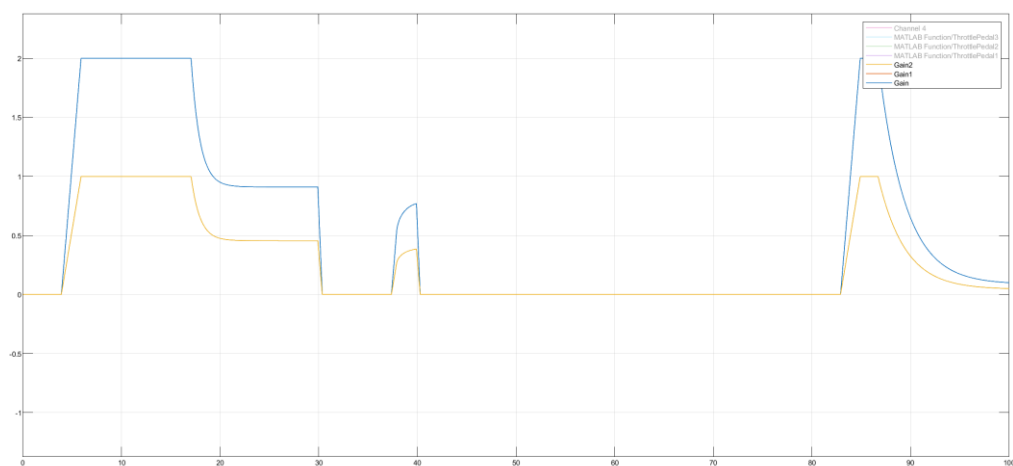
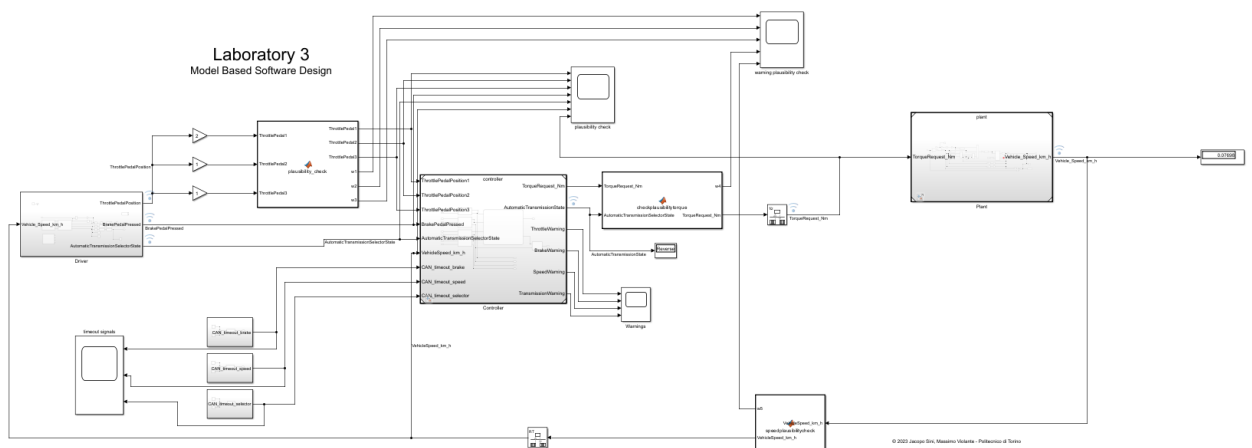
Implemented integration tests

Describe, in English, the scenarios tested at the integration level to verify the proper integration between the various units implementing the safety mechanisms.

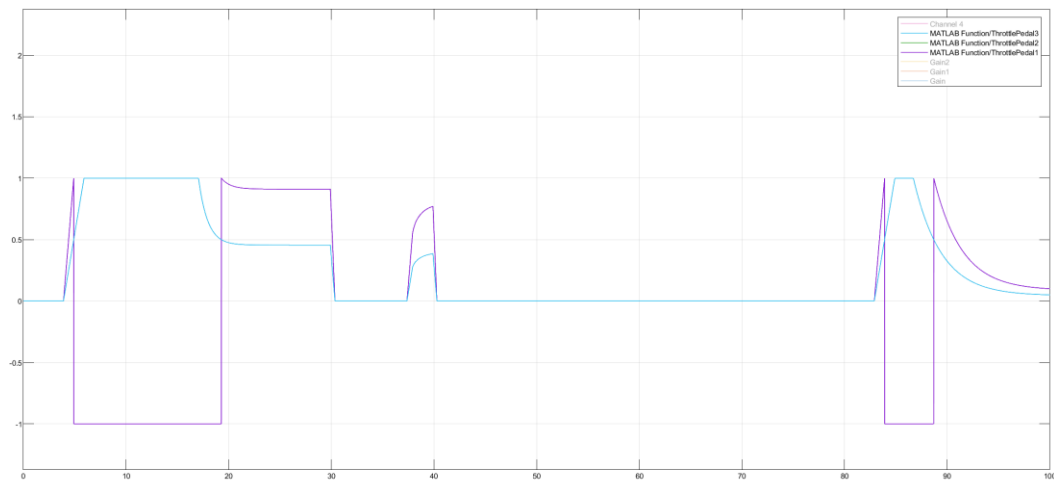
The goal of this testing phase was to verify system behavior under safety-critical fault conditions. Since unit tests had already covered all possible cases, integration testing focused on two representative fault scenarios.

These were implemented by injecting specific faults into the three throttle pedal channels:

- **Case 1: Single channel fault (channel 1):** The following plot shows the throttle pedal channel values after the introduction of disturbances. In this case, a gain of 2 was applied to the first channel in order to simulate a fault condition. As a consequence, the first channel (labelled *gain*) reaches values up to 2, which exceed the valid operating range $[0,1]$, thus correctly triggering the plausibility check.



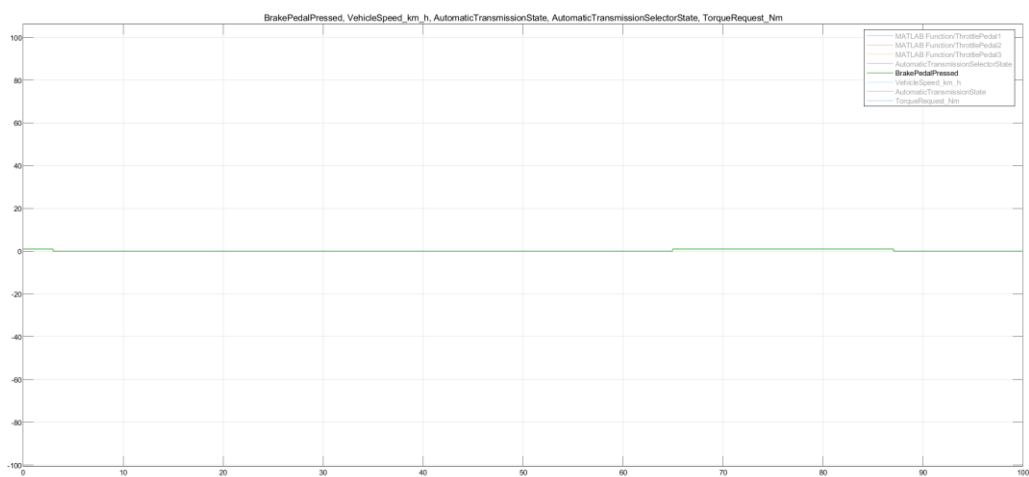
Throttle pedal signals before the plausibility check



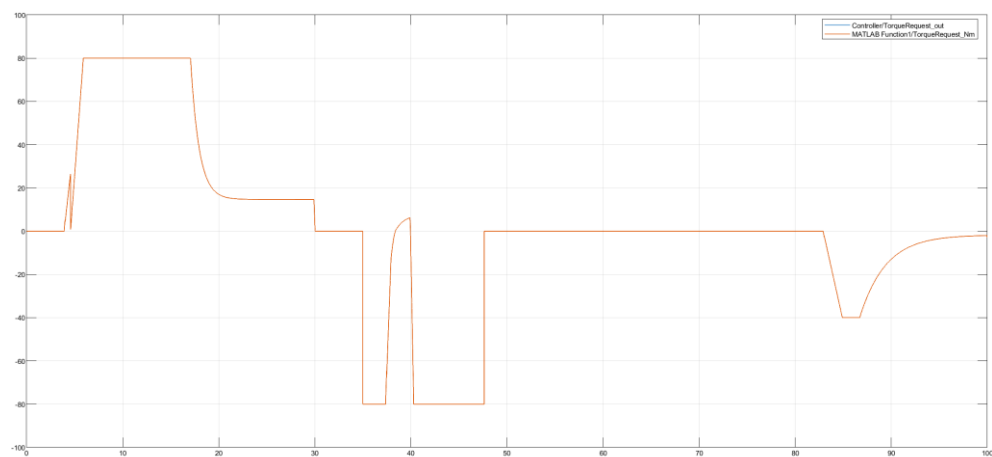
Throttle pedal signals after the plausibility check

After the plausibility check, we can observe that the first channel assumes a value of -1 in the regions where it exceeds the valid range, as defined in our implementation. This confirms that the plausibility check is functioning as intended.

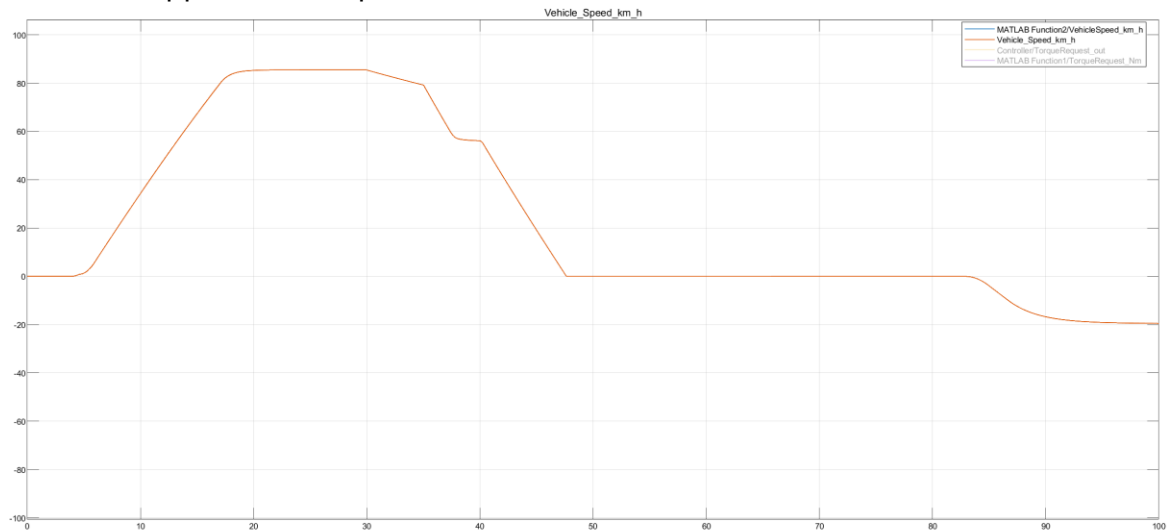
The brake pedal remains within the valid range, as shown in the plot below:



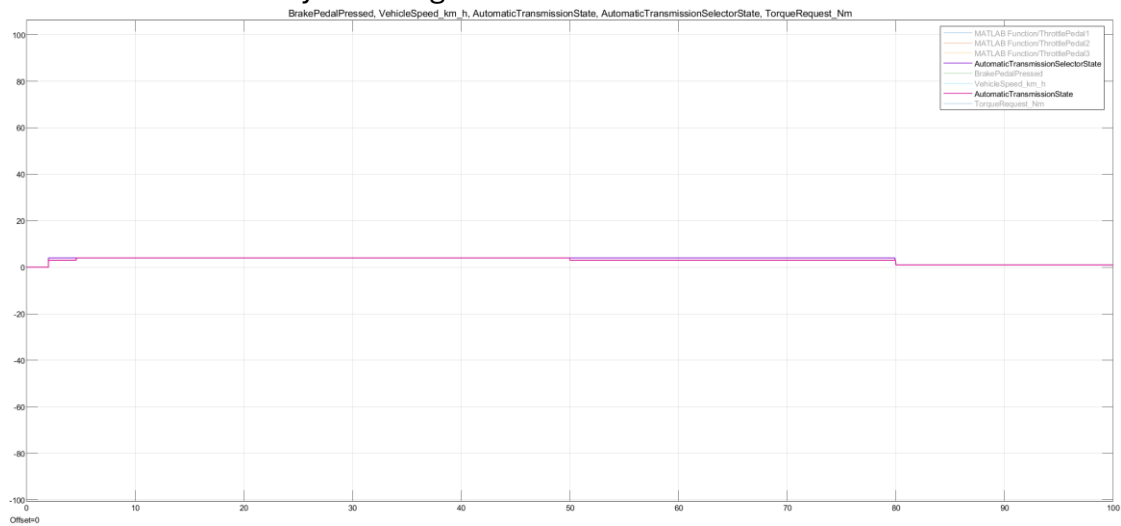
The graph below shows that the plausibility check does not affect the requested torque in this case:



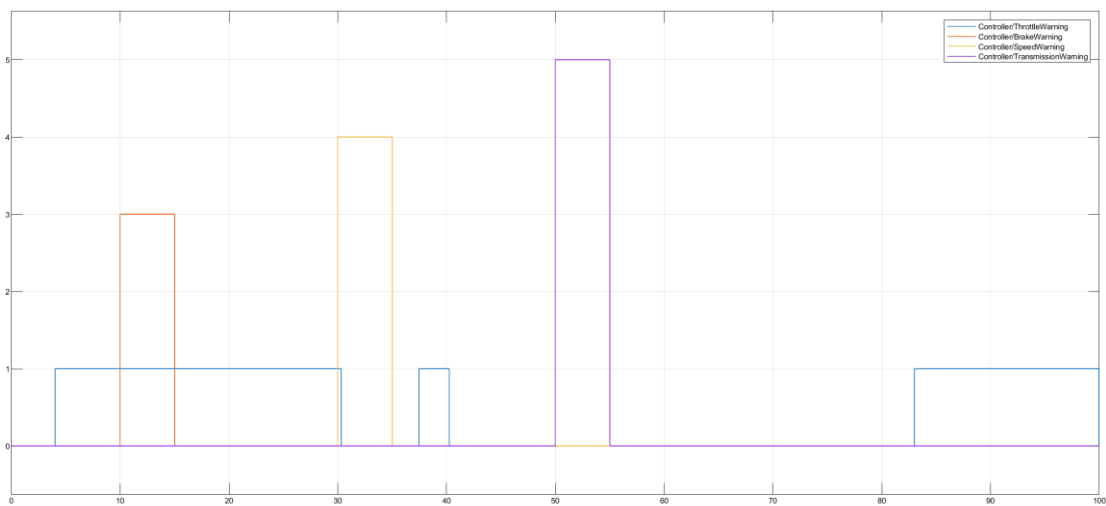
The same applies to the speed:



In particular, the automatic transmission operates in accordance with the safety constraints defined by the assigned ASIL level:



Warning signals for each malfunction were displayed on a scope, representing the driver's dashboard:

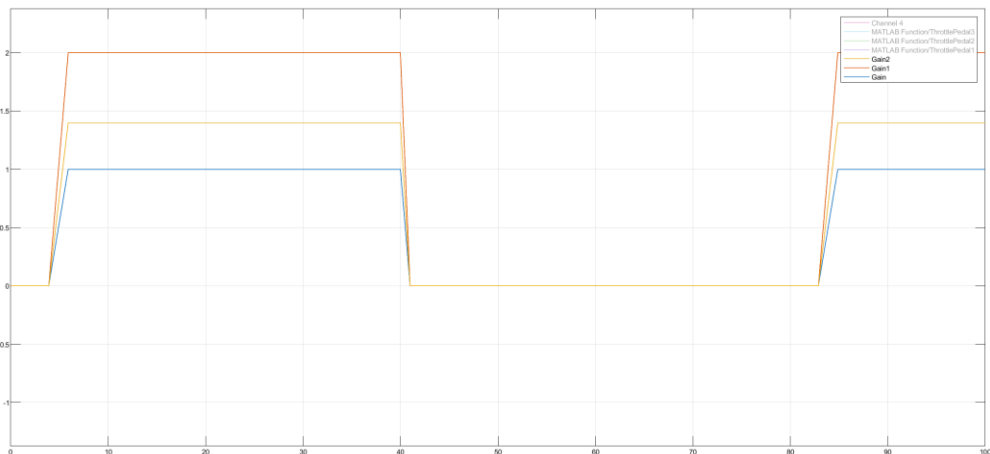
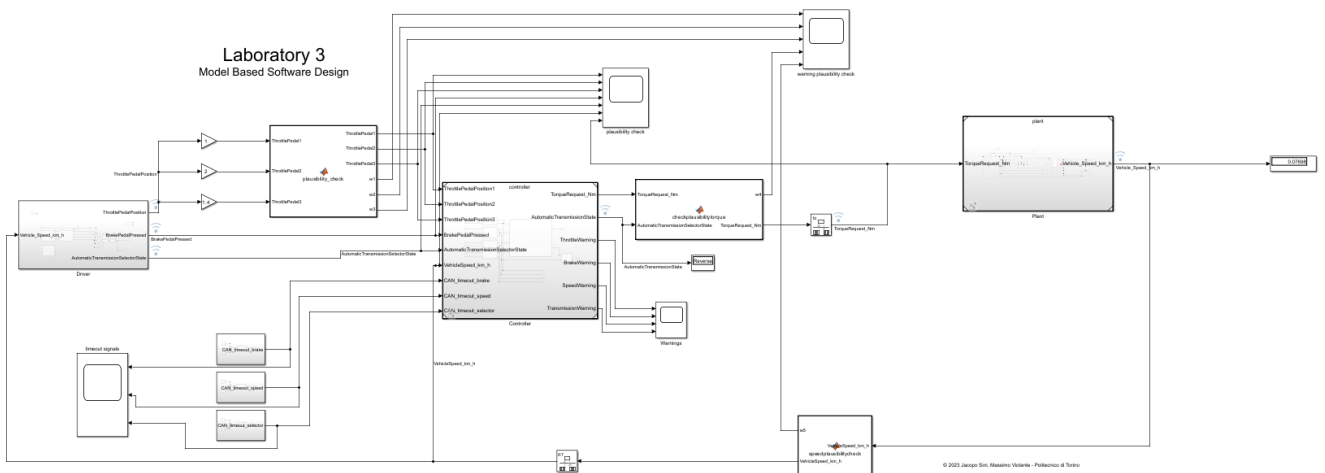


Warnings

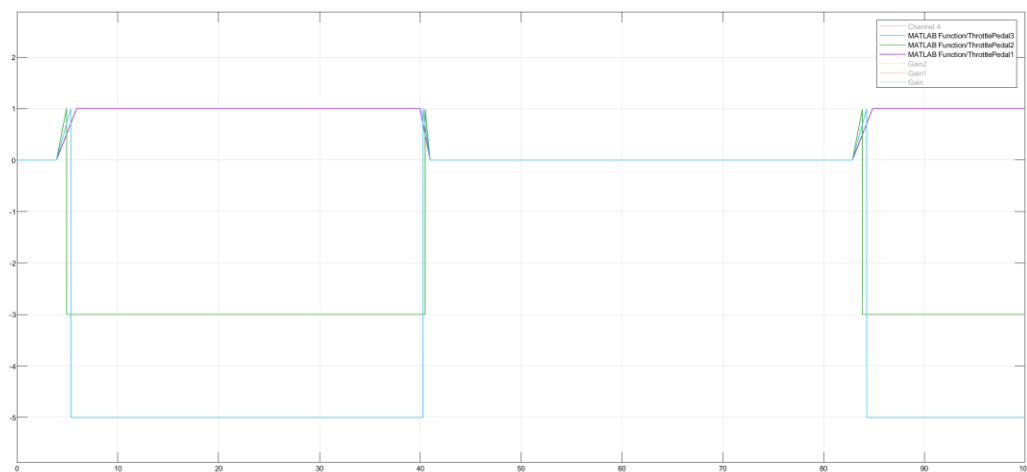
- **Case 2: Dual channel fault (channels 2 and 3)**

Disturbances were injected into the second and third throttle pedal channels to simulate a double fault scenario. According to the implemented safety logic, the system is expected to detect the absence of a valid majority, set the torque request to zero, and switch the driving mode to neutral.

The plot below displays the throttle pedal channel signals following the fault injection. In this test, a gain of 2 was applied to channel 2 and a gain of 1.4 to channel 3, effectively simulating signal deviation due to faults.

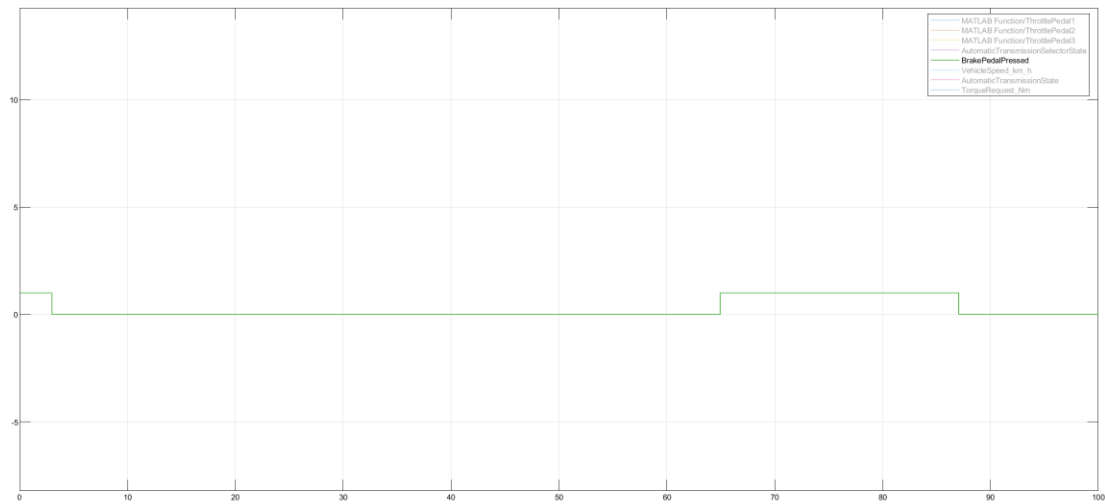


Initial Throttle Pedal Signals After Fault Injection

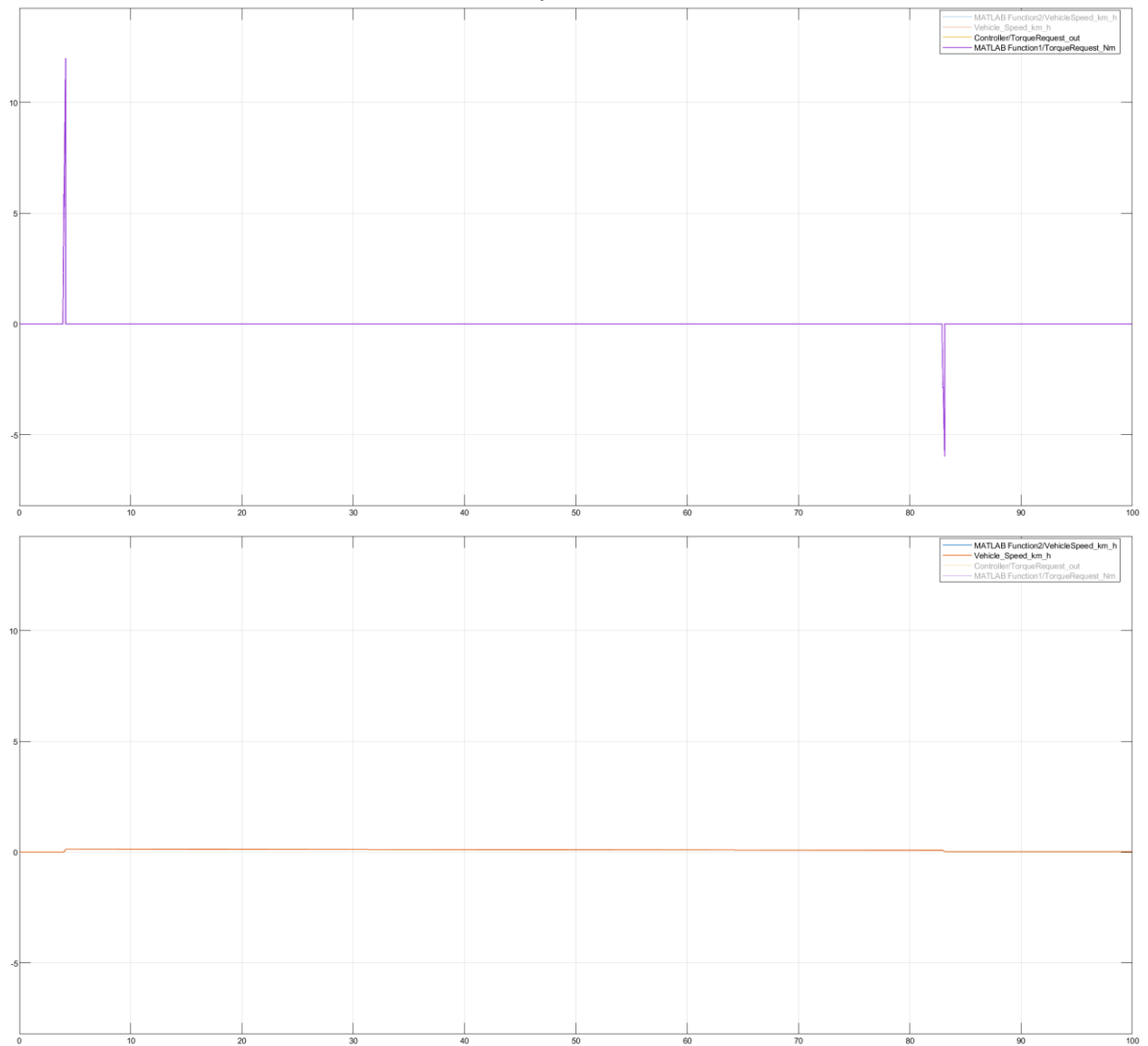


Throttle Pedal Values After Safety Validation

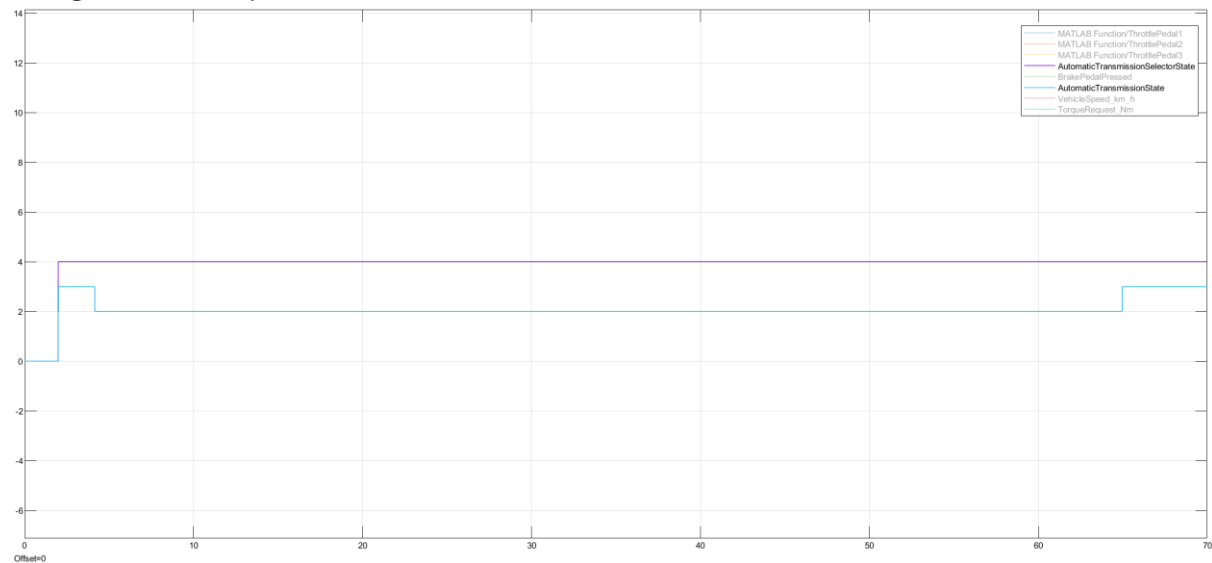
The brake pedal remains within the valid range, as shown in the plot below:



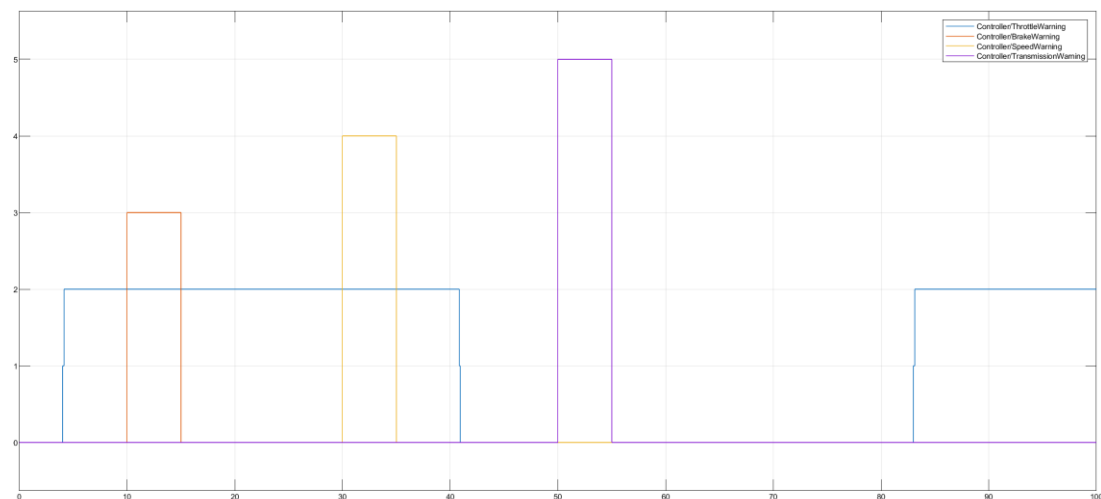
The graph below shows that the plausibility check does not alter the requested torque in this case; the same holds true for the speed.



The Automatic Transmission selector state, represented by the signal “AutomaticTransmissionSelectorState”, is compared with the final transmission state, “AutomaticTransmissionState”, to verify that the transition logic complies with the assigned ASIL requirements.



Warning signals corresponding to each malfunction were displayed on a scope:



Warnings

Conclusions about integration tests

The integration tests demonstrated that the system responds appropriately to critical fault conditions, ensuring compliance with the safety requirements. Additional scenarios were not tested at this level to avoid redundancy, as they were already validated through unit testing.